

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи № 9
з курсу “Життєвий цикл програмного забезпечення. Управління ІТ
проектами”
на тему: “Ведення документації у ІТ-проекті”

Виконав:

Студент. гр. ФеП-41 Фіняк Олег

Перевірив:

Франів В.А.

Львів – 2026

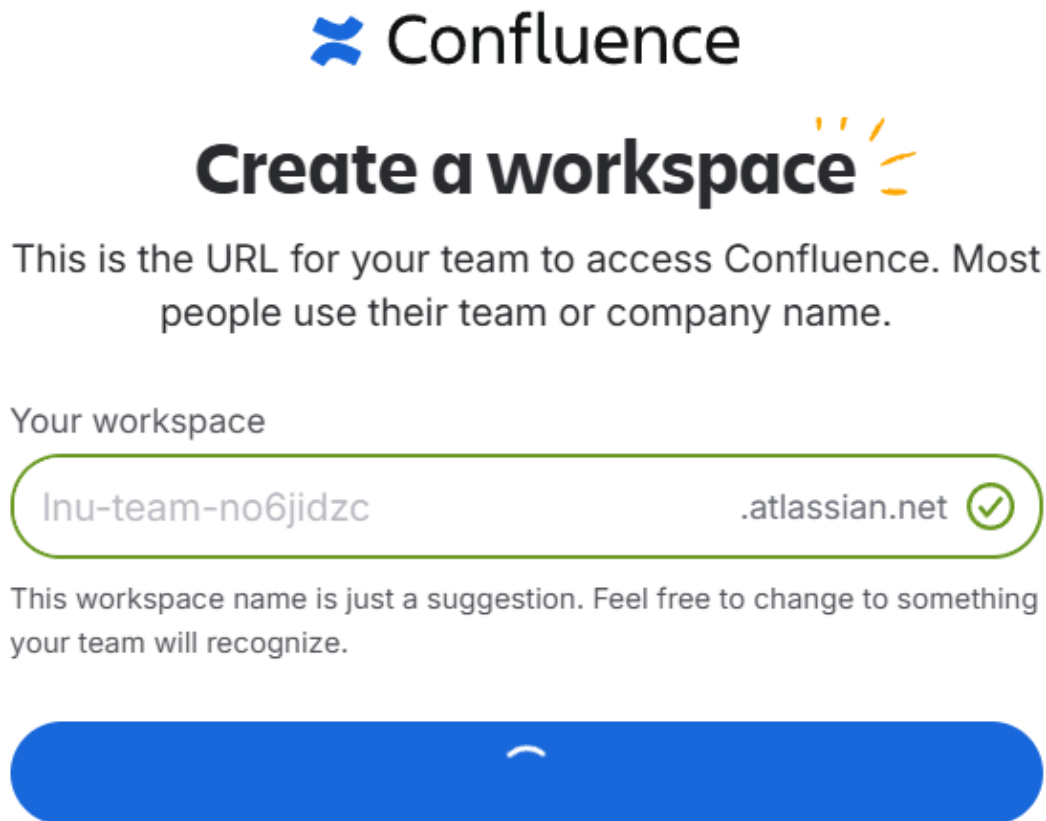
Мета роботи: Мета лабораторної роботи полягає в створенні найбільш поширених документів які супроводжуються ІТ проект.

Вимоги до результатів виконання лабораторної роботи:

1. Продемонструвати Atlassian Confluence workspace для обраного проекту.
2. Описати створений Atlassian Confluence workspace.
3. Проаналізувати плюси та мінуси використання Atlassian Confluence workspace як інструменту для ведення супроводжуючої документації.

Порядок виконання роботи:

1. Освоїти вище наведений теоретичний матеріал.
2. Ознайомитися з веб ресурсом Atlassian Confluence (якщо Ви ще не зареєструвалися, зареєструйтесь використовуючи університетську емейл адресу).



 Confluence

Create a workspace

This is the URL for your team to access Confluence. Most people use their team or company name.

Your workspace

This workspace name is just a suggestion. Feel free to change to something your team will recognize.



3. Для Вашого проекту створити Atlassian Confluence workspace та наповнити наступні сторінки: Project base line, загальна архітектура проекту (component diagram, відомості про репозиторії, CI/CD), основні бізнес процеси (sequence diagram), команда (відомості про команду).

Templates

Spaces

Apps

Oleh Finiak

Shortcuts

Content

Search by title

Explore Confluence Features

2026-05-03 Meeting ... DRAFT

Create

Blogs

Spaces

Filter spaces

Current

Oleh Finiak

Starred

My first space

Oleh Finiak

View all spaces

Create a space

Import from other tools



Educational Platform Home

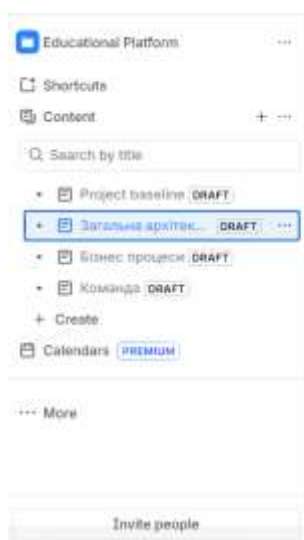
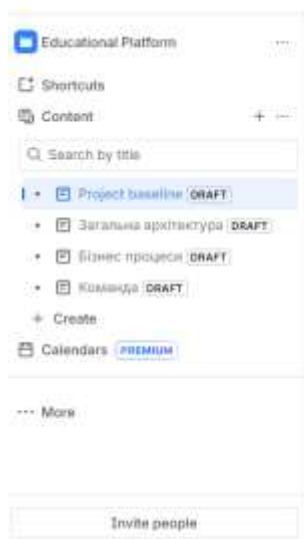
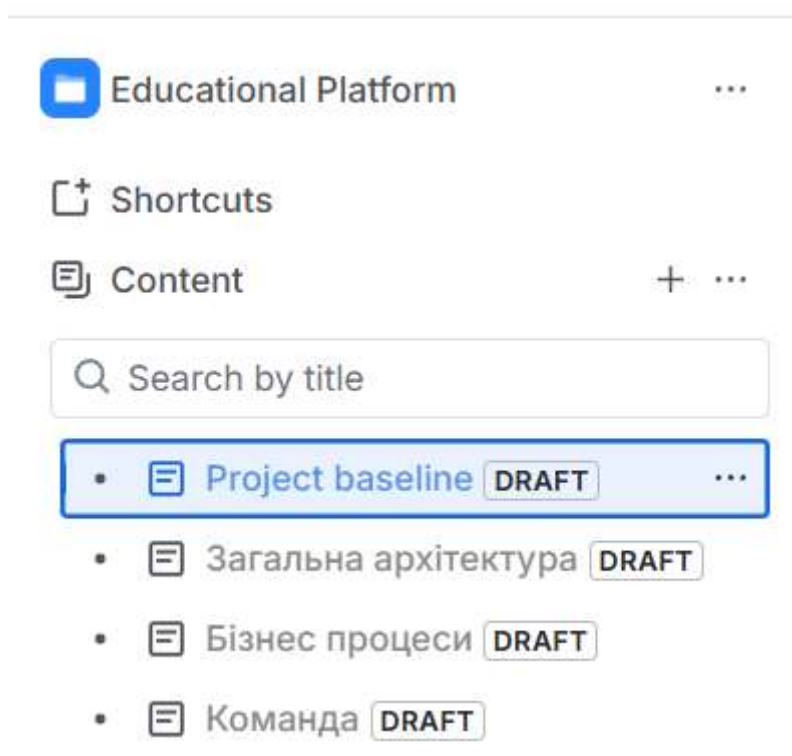
Welcome to your new space!

Spaces help your team structure, organize, and share work, so every team member has visibility into institutional knowledge and access to the information they need to do their best work.

Get started with the basics

Start editing this content:

- ☐ Click the pencil icon  or `e` on your keyboard to edit and start typing. You can edit anywhere.
- ☐ Hit `/` to see all the types of content you can add to your page. Try `/image` or `/table`
- ☐ Use the toolbar at the top to play around with *font*, *colors*, *formatting*, and more
- ☐ Click `close` to save your draft or `publish` when your page is ready to be shared



Project baseline

By Oleh Finko 2 min Listen

Обсяг робіт та вимоги (Scope)

Цей розділ визначає межі проекту, ключові цілі розробки та інфраструктурні вимоги до майбутньої платформи.

1.1. Опис та мета продукту

Розробка інтерактивної веб-платформи, призначеної для рекомендацій та структурування навчальних матеріалів для студентів природничих спеціальностей. Основна мета — спростити пошук релевантної академічної інформації завдяки класичній системі категоризації, тегування та прив'язки до навчальних планів (без використання ресурсоемних алгоритмів штучного інтелекту чи нейромереж).

1.2. Функціональні вимоги (Key Features)

Загальна архітектура

By Oleh Finko Listen

Репозиторій: [GitHub - Finko/Diploma](#)

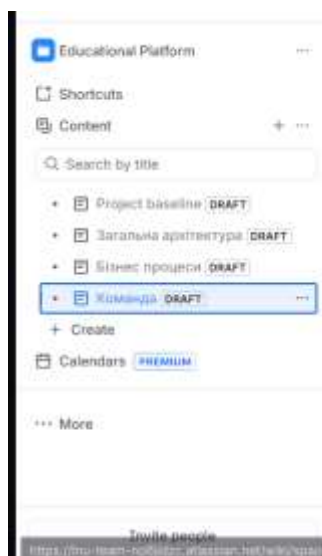
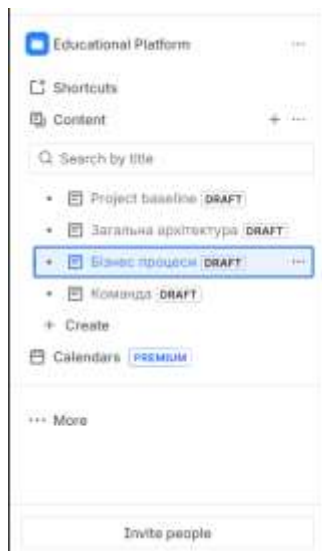
1. Високорівневий огляд системи (High-Level Architecture)

Освітня рекомендаційна платформа побудована за класичною клієнт-серверною архітектурою з використанням монолітного підходу для бекенд-частини. Система проектується як надійний, структурований веб-додаток, що забезпечує швидкий пошук, фільтрацію та рекомендацію навчальних матеріалів на основі реляційних зв'язків та метаданих, без залучення нейромереж.

Архітектурний патерн системи базується на **MVT (Model-View-Template)**, який є стандартом для обраного серверного фреймворку.

2. Серверна частина (Backend)

Ядром системи є веб-додаток, розроблений мовою програмування **Python 3.x** з використанням



Бізнес процеси

By Oleh Finko Listen

Модуль: Каталог та система рекомендацій

Актори: Користувач (Студент), Клієнтська частина (Frontend), Серверна частина (Django Backend), База даних (Database)

1. Опис бізнес-процесу

Головним бізнес-процесом освітньої платформи є забезпечення студента релевантними навчальними матеріалами на основі його запиту та атрибутів профілю. Процес ініціюється дією користувача в інтерфейсі (введення пошукового запиту або вибір фільтрів) і завершується динамічним оновленням сторінки з відображенням списку знайдених матеріалів.

Система працює за принципом асинхронної взаємодії (AJAX / Fetch API), що дозволяє отримувати дані без повного перезавантаження сторінки, забезпечуючи безперервний користувацький досвід (UX).

Команда

By Oleh Finko Listen

1. Загальна інформація про команду

Для успішної реалізації проекту освітньої рекомендаційної платформи залучена крос-функціональна IT-команда. Кожен учасник володіє спеціалізованими навичками, які покривають повний життєвий цикл розробки програмного забезпечення (SDLC) — від збору вимог та проєктування архітектури до тестування і розгортання в продуктове середовище. Команда працює за принципами гнучкої розробки, використовуючи систему контролю версій Git та практики CI/CD.

2. Ролі та зони відповідальності

Нижче наведено детальний опис кожної ролі, закріпленої за проєктом, ключові обов'язки та стек використовуваних інструментів.

2.1. System Architect (Системний архітектор)

Основні обов'язки: Стратегічний лідер технічної частини проєкту. Відповідає за високорівневе

4. Проаналізуйте інші інструменти для ведення супроводжувальної документації.

Висновок: У ході виконання лабораторної роботи було успішно досягнуто головну мету — отримано практичні навички створення ключових документів, які супроводжують життєвий цикл розробки програмного забезпечення в межах IT-проєкту. За допомогою корпоративного хмарного сервісу Atlassian Confluence було розгорнуто єдиний структурований робочий простір (Workspace) для дипломного проєкту освітньої рекомендаційної платформи. Цей простір об'єднав у собі базові параметри проєкту (Project Baseline), архітектурні рішення, алгоритми бізнес-процесів та організаційну структуру команди.

Практичний досвід доводить, що якісна та систематизована проєктна документація є не просто формальністю, а критично важливим інструментом для синхронізації роботи команди. Вона нівелює ризики втрати технічних знань (bus factor), значно прискорює та полегшує процес онбордингу нових розробників і слугує «єдиним джерелом правди» (Single Source of Truth) при прийнятті складних архітектурних чи управлінських рішень на всіх етапах життєвого циклу ПЗ.

Аналізуючи сучасний ринок платформ для ведення супроводжуючої документації, можна зробити наступні порівняльні висновки:

Atlassian Confluence: Зарекомендував себе як потужний корпоративний стандарт (Enterprise-рівня). Його головна перевага полягає у безшовній інтеграції з екосистемою Jira, суворому контролі прав доступу та підтримці складних макросів для інтеграції діаграм і схем. Проте інструмент має високий поріг входження та може виявитися занадто громіздким для мікро-проектів чи стартапів.

Notion: Виступає більш сучасною та швидкою альтернативою завдяки блочній архітектурі. Він пропонує неперевершену гнучкість у створенні баз даних та зв'язуванні сторінок, що робить його ідеальним для невеликих, динамічних команд та індивідуальної роботи. Однак йому бракує жорсткої стандартизації документів, яка є критичною у великих ІТ-корпораціях.

GitHub Wiki (Docs-as-Code): Найкращий вибір для суто технічної документації та Open Source проектів. Зберігання документації безпосередньо у репозиторії разом із вихідним кодом у форматі Markdown дозволяє версіонувати тексти через Git. Головний недолік — незручність використання для нетехнічних спеціалістів (наприклад, Project Manager або UI/UX Designer), оскільки відсутній звичний візуальний редактор (WYSIWYG).

Підсумок: Ведення вичерпної, завжди актуальної та добре структурованої проектної документації є обов'язковим індустріальним стандартом сучасної програмної інженерії. Вибір конкретного інструменту (Confluence, Notion чи GitHub Wiki) має залежати виключно від масштабу проекту, кваліфікації команди розробників та загальної корпоративної культури компанії.

Література:

- 21.Scott Chacon, Ben Straub. Pro Git. Apress. 2014. 456 p.
- 22.Jon Loeliger, Matthew McCullough. Version Control with Git. O'Reilly Media. 2012. 456 p.
- 23.Ryan Hodson. Git Essentials. Packt Publishing. 2015. 194 p.
- 24.Roger Dudler. Git Pocket Guide. O'Reilly Media. 2013. 234 p.
- 25.Seth Robertson, Steve Oualline. Practical Git: For Everyday Use. No Starch Press. 2014. 300 p.